

A06-Tokenization-Deep-Dive

1 Tokenization Deep Dive — Text, Image, and Audio

Document Version: 1.0

Generated: February 22, 2026

Standard: Tokenization Methods for Neural Network I/O

Status: Foundational Reference

Related: [01 - Neural Networks Fundamentals](#) (Tokenization section)

1.1 Table of Contents

1. [Overview](#)
 2. [Why Tokenization Matters](#)
 3. [Part 1 — Text Tokenization](#)
 4. [Part 2 — Image Tokenization](#)
 5. [Part 3 — Audio Tokenization](#)
 6. [Cross-Modal Tokenization](#)
 7. [Comparison Matrix](#)
 8. [References](#)
-

1.2 Overview

Tokenization is the **fundamental bridge** between raw sensory data (text, images, audio) and the numerical tensors that neural networks process. Every AI system — from GPT to Stable Diffusion to Suno — must convert its inputs and outputs through tokenization. This document covers the algorithms, formats, and practical implementations for all three major modalities.

1.2.1 Scope

- **Text:** BPE, WordPiece, SentencePiece, Unigram, tiktoken (detailed algorithms & examples)
- **Image:** Patch embedding (ViT), VQ-VAE/VQ-GAN (DALL-E, Stable Diffusion), CLIP tokenization
- **Audio:** Mel spectrograms, Whisper tokenization, neural audio codecs (Encodec/SoundStream), music composition tokenization (Suno, MusicGen)

1.2.2 Cross-Reference

For text tokenization fundamentals (BPE pseudocode, C++ implementations), see [01 - Neural Networks Fundamentals § Tokenization](#). This document extends that foundation with deeper algorithmic detail and covers image and audio modalities.

1.3 Why Tokenization Matters

1.3.1 The Core Problem

What are neural networks and what do they need?

Think of a neural network as a very sophisticated calculator that can only work with numbers arranged in perfectly rectangular grids (called **tensors**). Imagine a spreadsheet where every cell must contain a number, and every row must have exactly the same number of columns.

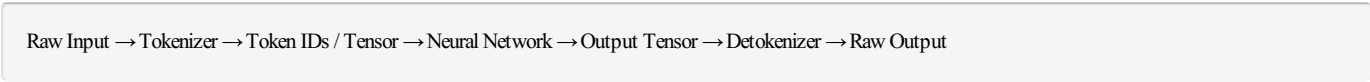
But the real world doesn't give us data in this neat format: - **Text:** We write sentences of different lengths — "Hi" has 2 letters, "How are you today?" has 18 characters. A neural network can't handle this variability directly. - **Images:** Photos come in all sizes — your phone might take 4000×3000 pixel photos, while an old website image might be 100×100 pixels. The neural network needs every image to have the same dimensions. - **Audio:** A song might be 3 minutes long, a voice message might be 10 seconds. The network needs consistent, fixed-size chunks to process.

Why is this a problem?

Imagine trying to do math on these mixed inputs: - You can't add "Hello" + "Hi" directly - You can't multiply a 100×100 image with a 4000×3000 image - You can't process a 10-second audio clip the same way as a 3-minute song

Neural networks need everything converted to the same “language” — numbers in consistent, rectangular arrangements.

1.3.2 The Solution: Tokenization



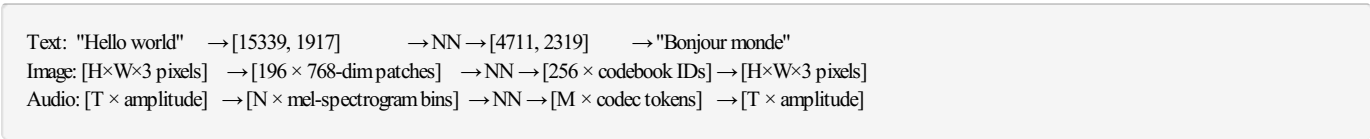
Plain-Language Intuition (Professional, Step-by-Step):

Think of tokenization as **labeling tiny pieces of input** so a model can look them up quickly and consistently:

- 1. **Split** the raw input into small pieces (characters, patches, or time frames).
- 2. **Map** each piece to a stable numeric ID (or compact vector).
- 3. **Feed** those IDs/vectors into the neural network, which only understands numbers.
- 4. **Reverse** the process (detokenize) to convert model outputs back to human-readable form.

This is why tokenization is so critical: it decides **what the model “sees”** and **how much detail is kept**.

Abstract Example:



1.3.3 Properties of Good Tokenization

Property	Text	Image	Audio
Reversible	Yes (lossless)	Approximately (lossy)	Approximately (lossy)
Fixed vocabulary	Yes (32K–200K tokens)	Yes (codebook 1K–16K)	Yes (codebook 1K–8K)
Handles unseen input	Subword fallback	Patch-based (any resolution)	Mel-based (any audio)
Compression ratio	~3–4 chars per token	~16×16 pixels per token	~320 samples per token

1.4 Part 1 — Text Tokenization

1.4.1 Plain-Language Walkthrough (What the Model Actually Sees)

Let’s take a simple sentence and walk it through tokenization in slow, explicit steps.

Input sentence:

"The cat sat."

Step 1 — Normalize and split (conceptual): - Keep punctuation as its own piece. - Preserve spaces (some tokenizers encode them explicitly).

One possible internal split:

"The" " " "cat" " " "sat" "."

Step 2 — Map pieces to token IDs:

"The" → 1024
" " → 220
"cat" → 5632
"sat" → 9011
"." → 13

Step 3 — What the model sees:

The model never sees words — it sees **numbers** that correspond to word-pieces. Different tokenizers differ mainly in **how they split** and **how the IDs are learned**.

1.4.2 Shared Principle + Key Nuances

All modern text tokenizers follow the same base pattern:

1. **Define a vocabulary** of subword pieces.
2. **Split new text** into those pieces.
3. **Replace pieces with IDs**.

The **nuances** are in how the vocabulary is constructed:

- **BPE**: merges the *most frequent* pairs.
- **WordPiece**: merges pairs that *maximize likelihood*.
- **SentencePiece (Unigram)**: starts large and **prunes** to the most useful pieces.
- **tiktoken**: BPE + a strict regex pre-tokenizer for consistency.

1.4.3 Algorithm 1: Byte Pair Encoding (BPE)

Used by: **GPT-2, GPT-3, GPT-4, LLaMA, Mistral, Grok**

1.4.3.1 What is BPE? (Non-Technical Explanation)

The Kitchen Analogy: Imagine you're a chef creating a cookbook, and you want to find the most efficient way to write recipes. Instead of writing "chopped onions" every time, you might create a shorthand "chop-onions". If you notice you often write "olive oil", you create "olive-oil" as a single unit. BPE works exactly like this — it finds the most common paired elements and combines them into single units.

The Real Process — Step by Step:

1. **Start with the smallest pieces** — individual letters or characters
2. **Look for the most frequently occurring pairs** — like how "e" and "d" often appear together as "ed"
3. **Combine the most common pairs into a single unit** — "e" + "d" becomes "ed"
4. **Repeat this process** until you have the desired number of vocabulary units
5. **The result**: A smart vocabulary that captures common patterns

1.4.3.2 Why This Works So Well

Problem: How do you handle words you've never seen before? - Traditional approach: Mark unknown words as "[UNKNOWN]" and lose all meaning - BPE approach: Break unknown words into known small pieces

Example: If the model has never seen "unhappiness" but knows "un", "happy", "ness", it can still understand the word!

1.4.3.3 The BPE Algorithm (Detailed and Accessible)

Input: A large collection of text (like all of Wikipedia) **Goal**: Create a vocabulary of useful word-pieces

Step-by-Step Process:

- STEP 1: Start with basic building blocks
- Begin with every unique character: {a, b, c, d, e, f, ..., space, !}
 - This is your initial "vocabulary"
- 2: Count all character pairs in your text
- How many times does "t"+"h" appear? (very often → "the", "that", "this")
 - How many times does "e"+"d" appear? (often → "ed", "red", "bed")
 - How many times does "i"+"n"+"g" appear? (very often → "ing")
- STEP 3: Find the most frequent pair
- Look through all counts and find the winner
 - Let's say "t"+"h" appears 50,000 times (most frequent)
- STEP 4: Merge this pair everywhere
- Replace all instances of "t" followed by "h" with a new single token "th"
 - Add "th" to your vocabulary
 - Now your text has fewer separate pieces
- STEP 5: Repeat until satisfied
- Now "th" might pair with "e" to become "the"
 - "in" might pair with "g" to become "ing"
 - Continue until you have enough vocabulary items (typically 32,000-50,000)

1.4.3.4 Technical Terms Explained

Corpus: The large collection of text used for training. Think of it as a massive library of books, articles, and websites that the algorithm studies to learn patterns.

Frequency: How often something appears. If “the” appears 1 million times in your text, it has a frequency of 1 million.

Adjacent Pairs: Characters or tokens that appear next to each other. In “hello”, the adjacent pairs are: (h,e), (e,l), (l,l), (l,o).

Vocabulary Size: The total number of different tokens the system can recognize. Larger vocabularies can capture more specific meanings but require more memory.

Concrete Example — Training BPE on a Tiny Corpus:

Input corpus: "low lower lowest"

Step	Vocabulary	Corpus (tokenized)	Most Frequent Pair
Init	{l, o, w, e, r, s, t, ·}	low · lower · lowest	(l, o) = 3
1	+ lo	low · lower · lowest	(lo, w) = 3
2	+ low	low · lower · lowest	(low, e) = 2
3	+ lowe	low · lower · lowest	(lowe, r) = 1, (lowe, s) = 1
4	+ lower	low · lower · lowest	...

Result: Words like “low”, “lower”, “lowest” share the prefix tokens, enabling the model to understand morphological relationships.

Concrete — tiktoken (OpenAI’s BPE Implementation):

```

import tiktoken

# GPT-4o tokenizer
enc = tiktoken.encoding_for_model("gpt-4o")

# Tokenize
text = "Hello, how are you doing today?"
tokens = enc.encode(text)
print(f"Token IDs: {tokens}")
# Output: [9906, 11, 1268, 527, 499, 3815, 3432, 30]

print(f"Token count: {len(tokens)}") # 8 tokens

# Decode back
decoded = enc.decode(tokens)
print(f"Decoded: {decoded}")
# Output: "Hello, how are you doing today?"

# Inspect individual tokens
for token_id in tokens:
    token_bytes = enc.decode_single_token_bytes(token_id)
    print(f" ID {token_id:5d} → {token_bytes}")
# Output:
# ID 9906 → b'Hello'
# ID 11 → b','
# ID 1268 → b' how'
# ID 527 → b' are'
# ID 499 → b' you'
# ID 3815 → b' doing'
# ID 3432 → b' today'
# ID 30 → b'?'

```

Concrete — Multilingual BPE (LLaMA 3):

```

# LLaMA 3 uses a 128K vocabulary BPE tokenizer
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("meta-llama/Meta-Llama-3-8B")

# English
en_tokens = tokenizer.encode("Machine learning is powerful")
print(f"English: {len(en_tokens)} tokens → {en_tokens}")
# ~5 tokens

# Japanese (BPE handles CJK via byte fallback)
jp_tokens = tokenizer.encode("□□□□□□□□")
print(f"Japanese: {len(jp_tokens)} tokens → {jp_tokens}")
# ~8-12 tokens (less efficient for CJK)

# Code
code_tokens = tokenizer.encode("def hello():\n    print('world')")
print(f"Code: {len(code_tokens)} tokens → {code_tokens}")
# BPE learns common code patterns as single tokens

```

1.4.4 Algorithm 2: WordPiece

Used by: **BERT, DistilBERT, ELECTRA**

1.4.4.1 WordPiece: The “Smart Frequency” Approach

The Restaurant Menu Analogy: Imagine you’re designing a restaurant menu and want to create “combo meals”. BPE would simply look at what customers order most often together. WordPiece is smarter — it looks at whether items REALLY belong together or just happen to be ordered frequently.

Example: - Customers often order “burger + fries” (they truly go together) - Customers also often order “burger + Tuesday” (just because it’s Burger Tuesday) - WordPiece can tell the difference!

1.4.4.2 How WordPiece Differs from BPE

BPE asks: “What pairs appear most often?” **WordPiece asks:** “What pairs are most *meaningful* together?”

The Key Insight: Just because two things appear together frequently doesn’t mean they belong together. WordPiece uses a smarter scoring system

1.4.4.3 The WordPiece Scoring System (Explained Simply)

The Mathematical Idea (in plain language):

How meaningful is combining A and B?
= (How often A and B appear together)
÷ (How often A appears alone × How often B appears alone)

What This Means: - **High Score:** A and B appear together much MORE than you’d expect by chance - **Low Score:** A and B just happen to appear together, but aren’t really related

Concrete Example:

Consider combining "un" + "happy":
- "unhappy" appears 1000 times in training data
- "un" appears alone 5000 times total
- "happy" appears alone 8000 times total

BPE score: Just count 1000 (frequency of "unhappy")
WordPiece score: $1000 \div (5000 \times 8000)$ = Much higher if they truly belong together

This helps WordPiece realize that "un" and "happy" form a meaningful unit.

1.4.4.4 Technical Terms Explained

Likelihood: The probability that some data would occur given a model. In simple terms, “how well does our tokenization explain the text we see?”

Scoring Function: A mathematical formula that assigns a number to each potential merger. Higher numbers = better mergers.

Frequency vs. Meaningful Frequency: - **Frequency:** How many times something happens - **Meaningful Frequency:** How many times something happens compared to what you’d expect by random chance

1.4.4.5 The WordPiece Tokenization Process (Step-by-Step)

The Greedy Matching Strategy: Think of this like trying to make the longest possible words in a word game, but working left-to-right.

Detailed Algorithm Explanation:

How to break down a word using WordPiece:

STEP 1: Start at the beginning of the word

- Look at the entire remaining word
- Try to find the longest possible match in your vocabulary

STEP 2: Use a "greedy" approach

- Always try the longest possible piece first
- If that doesn't work, try the second-longest
- Keep shrinking until you find a match

STEP 3: Mark continuation pieces

- If a piece isn't at the start of a word, add "##" prefix
- This tells the model "this piece continues the previous piece"

STEP 4: Move to the next part

- Start over with whatever's left of the word
- Repeat until the entire word is tokenized

STEP 5: Handle unknown pieces

- If no piece in your vocabulary matches, mark as "[UNK]" (unknown)

Detailed Example Walkthrough:

Word: "embeddings"

Vocabulary includes: ["em", "##bed", "##ding", "##s", "embed", "##dings"]

Step 1: Look at "embeddings"

- Try "embeddings" → Not in vocabulary
- Try "embedding" → Not in vocabulary
- Try "embed" → Found it! □

Result so far: ["embed"]

Remaining "dings"

Step 2: Look at "dings" (add ## since it's continuing)

- Try "##dings" → Found it! □

Final result: ["embed", "##dings"]

The ## Symbol Explained: - **“embed”** = This starts a new word - **“##dings”** = This continues the previous piece - Without ##: The model might think “embed” and “dings” are separate words - With ##: The model knows they belong together as “embeddings”

1.4.4.6 Key Advantages of WordPiece

1. **Better at Handling Word Relationships:** Recognizes that “run”, “##ning”, “##ner” are all related to running
2. **Fewer Unknown Words:** Can break down any word into known pieces
3. **Language Awareness:** Learns meaningful linguistic patterns, not just frequent patterns

Concrete Example:

```

from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

# Common word → single token
tokens = tokenizer.tokenize("running")
print(tokens) # ['running']

# Less common word → split with ## prefix
tokens = tokenizer.tokenize("embeddings")
print(tokens) # ['em', '##bed', '##ding', '##s']

# Full encode with special tokens
encoded = tokenizer.encode("Hello world", add_special_tokens=True)
print(encoded) # [101, 7592, 2088, 102]
# 101 = [CLS], 7592 = "hello", 2088 = "world", 102 = [SEP]

# Decode
print(tokenizer.decode(encoded)) # "[CLS] hello world [SEP]"

```

1.4.5 Algorithm 3: SentencePiece (Unigram Model)

Used by: **T5, ALBERT, XLNet, mBART, LLaMA 1/2**

1.4.5.1 What is SentencePiece? (The Swiss Army Knife of Tokenizers)

The Universal Translator Analogy: Imagine you're creating a universal communication system that works for any human language — English, Chinese, Arabic, hieroglyphics, even made-up languages. Most tokenizers make assumptions about how languages work (like “words are separated by spaces”). SentencePiece doesn't make any assumptions at all!

Key Features Explained:

1. **Language-Agnostic:**
 - **What this means:** Works the same way for ANY language
 - **Why it matters:** Chinese doesn't use spaces between words. Arabic reads right-to-left. Thai uses different word boundaries. SentencePiece handles all of these!
2. **No Pre-tokenization:**
 - **What this means:** Doesn't break text into “words” first
 - **Why it matters:** What counts as a “word” varies hugely between languages
3. **Raw Unicode Processing:**
 - **What this means:** Looks at the actual characters, not assumptions about language structure
 - **Why it matters:** Can handle emojis, code, mathematical symbols, any text

1.4.5.2 The “Unigram” Approach vs. BPE/WordPiece

BPE/WordPiece: Start small and build bigger (“bottom-up”) - Begin with characters, gradually merge them - Like building a house brick by brick

SentencePiece Unigram: Start big and trim down (“top-down”) - Begin with many possible word pieces, remove the least useful ones - Like sculpting a statue — start with a big block and carve away excess

1.4.5.3 Why the “Top-Down” Approach Works Better

The SentencePiece Unigram Algorithm (Step-by-Step)

The “Survival of the Fittest” Approach:

STEP 1: Create a massive list of candidate word pieces

- Extract every possible substring (up to 16 characters long)
- From "hello world" you'd get: ["h", "he", "hel", "hell", "hello", "e", "el", "ell", ...]
- This creates hundreds of thousands of candidates!

STEP 2: Assign "usefulness scores" to each candidate

- Count how often each piece appears in your training text
- Convert counts to probabilities (more frequent = higher probability)
- Example: "the" appears very often → high probability, "xyz" appears rarely → low probability

STEP 3: Test what happens if we remove each piece

- For each candidate, simulate removing it from the vocabulary
- Ask: "How much harder is it to encode text without this piece?"
- Some pieces are critical (removing them makes encoding much harder)
- Other pieces are redundant (removing them barely matters)

STEP 4: Remove the least important pieces

- Identify pieces whose removal causes the least "damage"
- Remove 20% of the least important pieces
- Keep the vocabulary manageable

STEP 5: Repeat until you have the right size

- Continue removing pieces until you reach your target vocabulary size (e.g., 32,000)
- The surviving pieces are the most efficient for encoding text!

1.4.5.4 Technical Terms Explained

Substring: Any continuous sequence of characters within a larger string. "ell" is a substring of "hello".

Probability: A number between 0 and 1 indicating how likely something is. Frequent word pieces get high probabilities.

Loss Increase: How much "worse" your tokenization becomes if you remove a particular piece. High loss increase = very important piece.

Viterbi Algorithm: A method for finding the best way to split text into tokens. Think of it as finding the "optimal path" through possible tokenizations.

Concrete Example:

```
import sentencepiece as spm

# Train a SentencePiece model
spm.SentencePieceTrainer.train(
    input='training_data.txt',
    model_prefix='my_tokenizer',
    vocab_size=32000,
    model_type='unigram', # or 'bpe'
    character_coverage=0.9995,
    pad_id=0,
    eos_id=1,
    bos_id=2,
    unk_id=3
)

# Load and use
sp = spm.SentencePieceProcessor()
sp.load('my_tokenizer.model')

# Encode
tokens = sp.encode("Hello world!", out_type=str)
print(tokens) # ['▯Hello', '▯world', '!']
# Note: ▯ represents a space (whitespace is a regular character)

ids = sp.encode("Hello world!", out_type=int)
print(ids) # [8774, 296, 5]

# Decode (perfectly reversible)
print(sp.decode(ids)) # "Hello world!"

# Multilingual — no special handling needed
jp = sp.encode("□□□□□□□", out_type=str)
print(jp) # ['□', '□□□□□', '□□'] (learned as whole words)
```

1.4.6 Algorithm 4: tiktoken (OpenAI’s Production BPE)

Used by: GPT-3.5, GPT-4, GPT-4o

Key Feature: Byte-level BPE with regex-based pre-tokenization for consistency.

Pre-tokenization regex (GPT-4):

```
# GPT-4 splits text using this regex BEFORE applying BPE:
import regex
pattern = r'"""(?:[sdmt]|ll|ve|re)|[^\r\n]{1,3}|(?:\s|[^\s\p{N}]|[\p{N}]{1,3})|(?![\s\p{N}])|(?![\r\n])|s*[\r\n]|s+(?!S)|s+"""'

# This ensures:
# - Contractions stay together: "don't" → ["don", "'t"]
# - Numbers split at 3 digits: "123456" → ["123", "456"]
# - Punctuation is separate: "hello!" → ["hello", "!"]
# - Whitespace is preserved deterministically
```

1.4.7 Special Tokens

All tokenizers use special tokens for structure:

Token	BPE (GPT)	WordPiece (BERT)	SentencePiece (T5)	Purpose
Start of text	< endoftext > (overloaded)	[CLS]	<s>	Sequence start
End of text	< endoftext >	[SEP]	</s>	Sequence end
Padding	—	[PAD]	<pad>	Batch alignment
Unknown	— (byte fallback)	[UNK]	<unk>	Unknown tokens

Token	BPE (GPT)	WordPiece (BERT)	SentencePiece (T5)	Purpose
Mask	—	[MASK]	—	MLM training
Tool call	< tool_call >	—	—	Function calling
Image	< image >	—	—	Vision input

1.5 Part 2 — Image Tokenization

Images must be converted to sequences or grids of tokens before neural networks can process them. There are two fundamentally different approaches:

Guiding Principle: How Do We Make Images “Readable” to Neural Networks?

The Jigsaw Puzzle Analogy: Imagine you need to send a photograph through a mail system that can only handle numbered packages, not actual images. You’d need to:

- 1. **Cut the image into pieces** (like jigsaw puzzle pieces)
- 2. **Number each piece** or describe it somehow
- 3. **Send the numbers/descriptions** instead of the actual image pieces
- 4. **Reconstruct the image** on the other end using the numbers

This is exactly what image tokenization does! But there are different strategies:

1.5.0.1 Three Main Strategies for Image Tokenization

Strategy 1: Patch Embedding (ViT Style) - The Approach: Cut image into fixed-size squares, describe each square with numbers - **Like:** Taking a photo, cutting it into 16×16 pixel squares, then writing a detailed description of each square - **Result:** Each square becomes a list of numbers (like “mostly blue, some white, bright”) - **Used for:** Understanding and analyzing images

Strategy 2: VQ-VAE/VQ-GAN (Discrete Tokens) - The Approach: Cut image into squares, then find the “closest match” from a pre-made visual dictionary - **Like:** Taking photo squares and saying “this looks most like pattern #42 from our visual dictionary” - **Result:** Each square becomes a single number (token ID) from the dictionary - **Used for:** Generating new images

Strategy 3: Latent Diffusion (Continuous Compression) - The Approach: Compress the entire image into a much smaller representation that captures its essence - **Like:** Writing a very detailed, compact description of the entire photo - **Result:** A smaller grid of continuous values that represents the whole image - **Used for:** High-quality image generation

1.5.0.2 Key Technical Terms Explained

Patch: A small, fixed-size square cut from an image. Like a puzzle piece, but always the same size (e.g., 16×16 pixels).

Embedding: A list of numbers that represents something. Instead of storing “red circle”, we might store [0.8, 0.1, 0.9, 0.3, ...] which means “red circle” to the neural network.

Discrete vs. Continuous: - **Discrete:** Like multiple choice answers (A, B, C, or D) - **Continuous:** Like fill-in-the-blank with any number

Codebook: A dictionary that maps numbers to visual patterns. Entry #42 might always mean “blue sky with clouds”.

Quantization: The process of converting continuous values to discrete choices. Like rounding 3.7 to 4.

The rest of this section first shows the **shared step-by-step logic** on a tiny image, then details each approach and its nuances.

1.5.1 Step-by-Step Micro Example — 4×4 Image → 2×2 Patches

Toy Input (grayscale 4×4 image):

```
I=[
  [ 1, 2, 3, 4],
  [ 5, 6, 7, 8],
  [ 9,10,11,12],
  [13,14,15,16]
]
```

Step 1 — Split into 2×2 patches (patch_size=2):

```
P1 = [[1, 2], [5, 6]]   P2 = [[3, 4], [7, 8]]
P3 = [[9, 10], [13, 14]] P4 = [[11, 12], [15, 16]]
```

Step 2 — Flatten each patch:

```
P1 → [1, 2, 5, 6]
P2 → [3, 4, 7, 8]
P3 → [9, 10, 13, 14]
P4 → [11, 12, 15, 16]
```

Step 3 — Map to token representation (choose one):

- **ViT-style (continuous embeddings):** Apply a linear projection $W \in \mathbb{R}^{2 \times 4}$ (toy example): Then:
 - P1 $\rightarrow [1+6, 2+5] = [7, 7]$
 - P2 $\rightarrow [3+8, 4+7] = [11, 11]$
 - P3 $\rightarrow [9+14, 10+13] = [23, 23]$
 - P4 $\rightarrow [11+16, 12+15] = [27, 27]$ These 2D vectors are the **patch embeddings** (tokens).
- **VQ-VAE-style (discrete IDs):** Each patch embedding is mapped to the **nearest codebook entry** $\rightarrow$ token IDs, e.g.:

```
P1 → 42, P2 → 7, P3 → 1337, P4 → 890
```

- **Latent diffusion (continuous grid):** The image is compressed by a VAE encoder into a smaller **latent grid**, e.g. $2 \times 2 \times C$ values, which serve as the model input.

1.5.2 Approach 1: Patch Embedding (Vision Transformer — ViT)

Used by: ViT, CLIP, GPT-4V, Gemini, Claude Vision, LLaVA

Core Idea: Divide the image into fixed-size patches (e.g., 16×16 pixels), flatten each patch into a vector, and project it into the model's embedding dimension.

Abstract Algorithm:

```
ALGORITHM: PatchTokenize(image, patch_size, embed_dim)

H, W, C = image.shape // e.g., 224×224×3
num_patches = (H / patch_size) × (W / patch_size) // e.g., (224/16)² = 196

patches = []
FOR y = 0 TO H STEP patch_size DO
  FOR x = 0 TO W STEP patch_size DO
    patch = image[y:y+patch_size, x:x+patch_size, :] // 16×16×3 = 768 values
    flat_patch = flatten(patch) // → 768-dim vector
    patches.append(flat_patch)
  END FOR
END FOR

// Linear projection to embedding dimension
patch_embeddings = LinearProject(patches, embed_dim) // → 196 × embed_dim

// Add positional encoding (so model knows spatial layout)
FOR i = 0 TO num_patches DO
  patch_embeddings[i] += position_embedding[i]
END FOR

// Prepend [CLS] token for classification
tokens = [cls_token] + patch_embeddings // → 197 × embed_dim

RETURN tokens
```

Concrete Example — ViT Patch Tokenization:

Input Image: $224 \times 224 \times 3$ (RGB, standard ImageNet size)

Patch Size: 16×16 pixels

Patches: $14 \times 14 = 196$ patches

Each patch: $16 \times 16 \times 3 = 768$ values (flattened)

Projection: $768 \rightarrow 768$ (ViT-Base) or $768 \rightarrow 1024$ (ViT-Large)

+ Positional embedding: 197×768 learnable vectors

Final input to Transformer: $[\text{CLS}, \text{P1}, \text{P2}, \dots, \text{P196}] = 197 \text{ tokens} \times 768 \text{ dims}$

```
import torch
import torch.nn as nn

class ViTPatchEmbedding(nn.Module):
    """Convert image to sequence of patch embeddings."""

    def __init__(self, img_size=224, patch_size=16, in_channels=3, embed_dim=768):
        super().__init__()
        self.patch_size = patch_size
        self.num_patches = (img_size // patch_size) ** 2 # 196

        # Linear projection of flattened patches
        # Implemented as a convolution with kernel_size = stride = patch_size
        self.projection = nn.Conv2d(
            in_channels, embed_dim,
            kernel_size=patch_size, stride=patch_size
        )

        # Learnable [CLS] token
        self.cls_token = nn.Parameter(torch.randn(1, 1, embed_dim))

        # Positional embeddings (197 = 1 CLS + 196 patches)
        self.position_embedding = nn.Parameter(
            torch.randn(1, self.num_patches + 1, embed_dim)
        )

    def forward(self, x):
        # x shape: (batch, 3, 224, 224)
        B = x.shape[0]

        # Extract patches via convolution
        # (B, 3, 224, 224) → (B, 768, 14, 14)
        x = self.projection(x)

        # Reshape to sequence
        # (B, 768, 14, 14) → (B, 768, 196) → (B, 196, 768)
        x = x.flatten(2).transpose(1, 2)

        # Prepend CLS token
        cls_tokens = self.cls_token.expand(B, -1, -1)
        x = torch.cat([cls_tokens, x], dim=1) # (B, 197, 768)

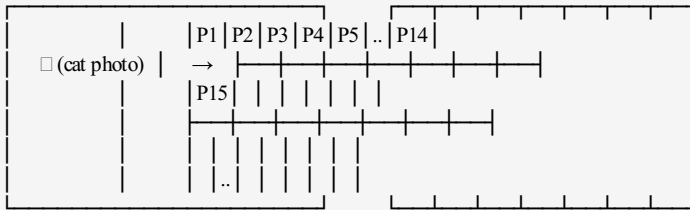
        # Add positional embeddings
        x = x + self.position_embedding

        return x # (B, 197, 768) — ready for Transformer layers

# Usage
patch_embed = ViTPatchEmbedding()
image = torch.randn(1, 3, 224, 224) # Random image
tokens = patch_embed(image)
print(f"Image tokenized to: {tokens.shape}") # (1, 197, 768)
```

Visualization — How ViT Sees an Image:

Original Image (224×224): Patch Grid (14×14 = 196 patches):



Each patch P_i : $16 \times 16 \times 3 = 768$ floats $\rightarrow$ projected to 768-dim embedding

Final: [CLS, P_1 , P_2 , ..., P_{196}] fed to Transformer (same as text tokens!)

1.5.3 Approach 2: VQ-VAE / VQ-GAN (Discrete Visual Tokens)

Used by: **DALL-E 1, Stable Diffusion (latent), Parti, Muse, VideoGPT**

Core Idea: Train an encoder to compress images into a **discrete codebook** of visual tokens. Like text tokenization but for images — each “token” represents a visual concept (edge, texture, color pattern).

Abstract Algorithm:

ALGORITHM: VQ-VAE_Tokenize(image, codebook)

```
// 1. Encode image to continuous latent space
z = Encoder(image) // (H, W, 3)  $\rightarrow$  (h, w, d) e.g., (224,224,3)  $\rightarrow$  (14,14,256)

// 2. Quantize: map each latent vector to nearest codebook entry
FOR each spatial position (i, j) DO
  distances =  $\|z[i,j] - \text{codebook}[k]\|$  for all k
  token_id[i,j] = argmin(distances)
  z_quantized[i,j] = codebook[token_id[i,j]]
END FOR

// 3. Result: grid of discrete token IDs
RETURN token_ids // (h, w) grid of integers, e.g., 14×14 = 196 token IDs
```

ALGORITHM: VQ-VAE_Detokenize(token_ids, codebook)

```
// 1. Look up codebook vectors
FOR each token_id DO
  z_quantized[i,j] = codebook[token_id]
END FOR

// 2. Decode back to image
image = Decoder(z_quantized) // (h, w, d)  $\rightarrow$  (H, W, 3)

RETURN image
```

Concrete Example — DALL-E 1 Style Discrete Tokenization:

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class VectorQuantizer(nn.Module):
    """Vector quantization layer — converts continuous to discrete tokens."""

    def __init__(self, num_embeddings=8192, embedding_dim=256):
        super().__init__()
        self.num_embeddings = num_embeddings #codebook size
        self.embedding_dim = embedding_dim

        # Codebook: 8192 visual "words"
        self.codebook = nn.Embedding(num_embeddings, embedding_dim)
        self.codebook.weight.data.uniform_(-1/num_embeddings, 1/num_embeddings)

    def forward(self, z):
        # z shape: (B, D, H, W) — encoder output
        B, D, H, W = z.shape

        # Reshape: (B, D, H, W) → (B*H*W, D)
        z_flat = z.permute(0, 2, 3, 1).reshape(-1, D)

        # Compute distances to all codebook entries
        # ||z - e||² = ||z||² + ||e||² - 2⟨z, e⟩
        distances = (
            z_flat.pow(2).sum(dim=1, keepdim=True)
            + self.codebook.weight.pow(2).sum(dim=1)
            - 2 * z_flat @ self.codebook.weight.t()
        )

        # Find nearest codebook entry
        token_ids = distances.argmin(dim=1) # (B*H*W,)

        # Look up quantized vectors
        z_quantized = self.codebook(token_ids)
        z_quantized = z_quantized.reshape(B, H, W, D).permute(0, 3, 1, 2)

        # Reshape token IDs to spatial grid
        token_grid = token_ids.reshape(B, H, W)

        return z_quantized, token_grid

    def decode_tokens(self, token_ids):
        """Convert token IDs back to embeddings for decoder."""
        return self.codebook(token_ids)

# Example usage
vq = VectorQuantizer(num_embeddings=8192, embedding_dim=256)
encoder_output = torch.randn(1, 256, 16, 16) # Simulated encoder output

z_quantized, token_ids = vq(encoder_output)
print(f"Image encoded to {token_ids.shape} discrete tokens") # (1, 16, 16)
print(f"Token vocabulary size: {vq.num_embeddings}") # 8192
print(f"Total tokens per image: {16*16} = 256")
print(f"Example tokens: {token_ids[0, :4, :4]}")
# Output: tensor([[ 42, 1337, 891, 4200],
#                [123, 567, 890, 1111],
#                ...])

```

How DALL-E 1 Uses Discrete Image Tokens:

Text: "a cat sitting on a mat"
↓ (Text tokenizer — BPE)
Text tokens: [64, 3797, 5765, 319, 64, 2603]
↓ (Autoregressive Transformer generates)
Image tokens: [42, 1337, 891, ..., 4200] (256 tokens from 8192-entry codebook)
↓ (VQ-VAE Decoder)
Image: 256×256 RGB image of a cat on a mat

1.5.4 Approach 3: Latent Diffusion Tokenization (Stable Diffusion)

Used by: **Stable Diffusion, SDXL, DALL-E 3, Midjourney**

Unlike VQ-VAE (discrete tokens), Stable Diffusion uses a **continuous latent space** via a VAE:

Image (512×512×3) → VAE Encoder → Latent (64×64×4) → Diffusion → Latent → VAE Decoder → Image

Compression ratio: 512×512×3 = 786,432 values → 64×64×4 = 16,384 values (48× compression)

```
# Stable Diffusion latent encoding (simplified)
from diffusers import AutoencoderKL

vae = AutoencoderKL.from_pretrained("stabilityai/sd-vae-ft-mse")

# Encode image to latent space
image_tensor = torch.randn(1, 3, 512, 512) # Input image
latent = vae.encode(image_tensor).latent_dist.sample()
print(f"Latent shape: {latent.shape}") # (1, 4, 64, 64)

# Decode latent back to image
decoded = vae.decode(latent).sample
print(f"Decoded shape: {decoded.shape}") # (1, 3, 512, 512)
```

1.6 Part 3 — Audio Tokenization

Audio tokenization is crucial for speech recognition (Whisper), text-to-speech (TTS), music generation (Suno, MusicGen), and audio analysis.

Guiding Principle: Converting Sound Waves to “Digital Language”

The Music Sheet Analogy: Imagine you’re a musician who needs to convert any song (whether it’s a guitar solo, human speech, or bird chirping) into sheet music that any other musician can read and play back.

The Challenge: Raw audio is like a constantly wiggling line (sound wave) that goes up and down thousands of times per second. Neural networks need this converted into organized, structured information.

The Universal Solution: All audio tokenizers follow the same basic approach:

1. **Slice Time into Small Windows:** Like taking snapshots of the sound wave every few milliseconds
2. **Analyze Each Window:** Ask “What frequencies (pitches) are present in this tiny time slice?”
3. **Create a Time-Frequency Map:** Build a chart showing “at each moment in time, which frequencies were loud or quiet”
4. **Either Keep It Continuous or Make It Discrete:**
 - **Continuous:** Keep the exact loudness values (good for recognition)
 - **Discrete:** Round to specific “audio vocabulary” entries (good for generation)

1.6.0.1 Two Main Strategies for Audio Tokenization

Strategy 1: Continuous Spectrograms (Whisper Style) - The Approach: Create detailed time-frequency charts (spectrograms) and keep precise values - **Like:** Making detailed sheet music with exact volume markings for every note - **Result:** A grid of continuous numbers representing frequency energy over time - **Used for:** Speech recognition, audio analysis - **Example:** Whisper mel spectrograms

Strategy 2: Discrete Audio Tokens (Codec Style) - The Approach: Convert audio into sequences of discrete “audio words” from a learned vocabulary - **Like:** Creating a musical language where each “word” represents a specific type of sound pattern - **Result:** Sequences of token IDs (like text tokens but for audio) - **Used for:** Audio generation, music synthesis - **Example:** Encodec, SoundStream, Bark

1.6.0.2 Key Technical Terms Explained

Waveform: The raw audio signal — a line that goes up and down representing air pressure changes over time. What your microphone captures.

Time-Frequency Representation: A chart showing which frequencies (itches) are present at each moment in time. Like a heatmap of sound.

Spectrogram: A visual representation of audio showing frequency content over time. Looks like a colorful chart with time on one axis and pitch on the other.

FFT (Fast Fourier Transform): A mathematical tool that analyzes a chunk of audio and tells you which frequencies are present. Like analyzing a chord and saying “this contains C, E, and G notes.”

Mel Scale: A frequency scale that matches human hearing better than linear frequency. We hear low frequencies more distinctly than high ones.

Quantization: Converting continuous values into discrete choices. Like rounding precise measurements to the nearest standard size.

1.6.1 Step-by-Step Micro Example — 8-Sample Waveform → Toy Spectrogram

Toy Waveform (8 samples):

```
x = [0, 1, 0, -1, 0, 1, 0, -1]
```

Step 1 — Windowing (window_size=4, hop=2):

```
F0 = [0, 1, 0, -1] (samples 0-3)
F1 = [0, -1, 0, 1] (samples 2-5)
F2 = [0, 1, 0, -1] (samples 4-7)
```

Step 2 — FFT Magnitudes (toy illustration): For this symmetric pattern, the magnitude spectrum is similar across frames:

```
|FFT(F0)| ≈ [0, 2, 0, 2]
|FFT(F1)| ≈ [0, 2, 0, 2]
|FFT(F2)| ≈ [0, 2, 0, 2]
```

Step 3 — Apply a tiny 2-bin “mel” filterbank (low/high):

```
Low bin = 2, High bin = 2 (for each frame)
```

Step 4 — Log scale (toy values):

```
log_mel ≈ [log10(2), log10(2)] for each frame
```

Result — Toy 2×3 Spectrogram:

```
mel = [
  [0.301, 0.301, 0.301], # Low bin across frames
  [0.301, 0.301, 0.301] # High bin across frames
]
```

In real systems, this becomes an $80 \times T$ mel spectrogram (Whisper) or a higher-resolution feature map that is then **quantized into discrete codec tokens** (Encoder/SoundStream) for generation tasks.

1.6.2 Step 1: Waveform to Spectrogram (The Foundation Process)

All audio AI systems first convert raw waveforms to a **time-frequency representation**. Think of this as converting a wiggly line (sound wave) into a detailed chart.

1.6.2.1 The Mel Spectrogram Process — Step by Step

The Concert Hall Analogy: Imagine you're sitting in a concert hall with a special hearing device that can tell you exactly which instruments are playing at each moment, and how loud each one is.

Detailed Algorithm (Human-Friendly):

STEP 1: Take "Audio Snapshots" (Short-Time Fourier Transform)

- Slide a "listening window" across the audio, a few milliseconds at a time
- At each position, analyze what frequencies (pitches) are present
- Like taking a photo of the sound every 10 milliseconds

Technical details:

- Window size: Usually 25 milliseconds (400 samples at 16kHz)
- Step size: Usually 10 milliseconds (160 samples at 16kHz)
- This creates overlapping snapshots for smooth analysis

STEP 2: Analyze Each Snapshot (FFT)

- For each audio snapshot, use math (FFT) to break it down into frequencies
- Ask: "How much energy is at 100Hz? 200Hz? 1000Hz? etc."
- Result: A spectrum showing which pitches were loud or quiet in that snapshot

What we get:

- Each snapshot becomes a list of frequency energies
- Low numbers = low pitches (bass), high numbers = high pitches (treble)
- Energy values tell us how loud each frequency was

STEP 3: Convert to Human-Hearing Scale (Mel Filtering)

- Humans don't hear all frequencies equally well
- We're more sensitive to differences in low frequencies than high frequencies
- Apply "mel filters" that group frequencies the way humans actually hear them

Why this matters:

- Linear frequency: 100Hz, 200Hz, 300Hz, 400Hz... (equal spacing)
- Mel frequency: Groups more like how our ears actually work
- More detail in bass range, less detail in treble range

STEP 4: Make It Logarithmic (Human-Like Volume)

- Human hearing is logarithmic (we hear ratios, not absolute differences)
- Converting 1→2 times louder feels the same as 10→20 times louder
- Take the logarithm so the numbers match human perception

STEP 5: Final Result

- A mel spectrogram: a grid of numbers
- Rows = different frequency bands (mel bins, usually 80 total)
- Columns = time frames (usually 100 frames per second)
- Each number = how much energy was in that frequency band at that time

1.6.2.2 Technical Terms Explained

Short-Time Fourier Transform (STFT): The process of analyzing small chunks of audio to see what frequencies are in each chunk. Like examining each paragraph of a book separately.

FFT (Fast Fourier Transform): A mathematical algorithm that takes a chunk of audio and tells you which frequencies (pitches) are present and how strong they are.

Hann Window: A mathematical function applied to each audio chunk to reduce artifacts. It smoothly fades the edges of each chunk to zero.

Power Spectrum: Shows how much energy is at each frequency. Calculated by squaring the FFT results.

Mel Scale: A perceptually-motivated frequency scale that better matches human hearing. Lower frequencies get more resolution.

Logarithmic Scale: Converts energy values to match how humans perceive loudness. Makes quiet and loud sounds more distinguishable.

Epsilon: A tiny number added before taking the logarithm to prevent mathematical errors (can't take log of zero).

Concrete Example — Whisper-Style Mel Spectrogram:

```

import torch
import torchaudio

def audio_to_mel_spectrogram(audio_path: str,
                              sample_rate: int = 16000,
                              n_mels: int = 80,
                              hop_length: int = 160,
                              n_fft: int = 400):
    """Convert audio file to log-mel spectrogram (Whisper format)."""

    # Load audio
    waveform, sr = torchaudio.load(audio_path)

    # Resample if needed
    if sr != sample_rate:
        resampler = torchaudio.transforms.Resample(sr, sample_rate)
        waveform = resampler(waveform)

    # Compute mel spectrogram
    mel_transform = torchaudio.transforms.MelSpectrogram(
        sample_rate=sample_rate,
        n_fft=n_fft,      # FFT window size (25ms at 16kHz)
        hop_length=hop_length, # Step size (10ms at 16kHz)
        n_mels=n_mels      # Number of mel frequency bins
    )

    mel_spec = mel_transform(waveform) # (1, 80, num_frames)

    # Log-scale
    log_mel = torch.log10(torch.clamp(mel_spec, min=1e-10))

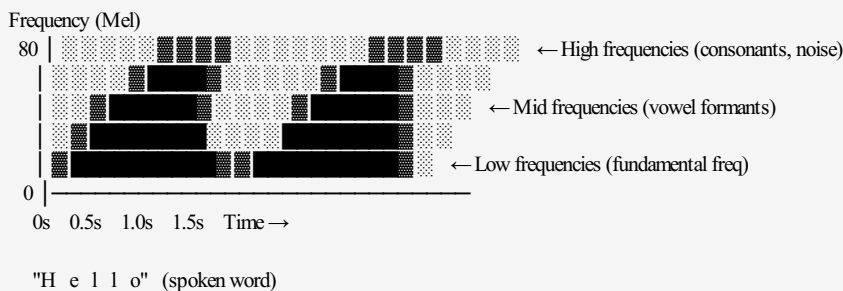
    # Normalize (Whisper-style)
    log_mel = torch.clamp(log_mel, min=log_mel.max() - 8.0)
    log_mel = (log_mel + 4.0) / 4.0 # Normalize to ~[0, 1]

    return log_mel

# Example
mel = audio_to_mel_spectrogram("speech.wav")
print(f"Mel spectrogram shape: {mel.shape}")
# For 10 seconds of audio at 16kHz:
# (1, 80, 1000) — 80 mel bins × 1000 time frames (10ms each)

```

Visualization — What a Mel Spectrogram Looks Like:



1.6.3 Step 2: Audio Tokenization Methods

1.6.3.1 Method A: Whisper-Style (Speech Recognition)

Used by: **OpenAI Whisper**, **Google USM**, **Meta SeamlessM4T**

Whisper treats audio as a mel spectrogram and processes it like a sequence with a Transformer encoder — no explicit discrete tokenization of audio.

Architecture:

Audio (30s max) → Mel Spectrogram (80×3000) → 2 Conv Layers → Transformer Encoder → Cross-Attention → Text Tokens

```
import whisper

# Load model
model = whisper.load_model("base")

# Transcribe
result = model.transcribe("speech.wav")
print(result["text"])
# Output: "Hello, how are you doing today?"

# Access internal mel spectrogram
audio = whisper.load_audio("speech.wav")
mel = whisper.log_mel_spectrogram(audio)
print(f"Mel shape: {mel.shape}") # (80, 3000) — 30 seconds
```

1.6.3.2 Method B: Neural Audio Codecs (Discrete Audio Tokens)

Used by: **Meta Encodec, Google SoundStream, Bark, VALL-E, MusicGen, Suno**

1.6.3.3 What Are Neural Audio Codecs? (The Audio Dictionary Approach)

The Universal Music Language Analogy: Imagine creating a universal language for describing any sound using only 1,024 “audio words”. Each word represents a specific type of sound pattern — like “crisp drum hit”, “warm guitar strum”, “human vowel ‘ah’”, etc.

The Key Insight: If we can convert any audio into sequences of these standardized “audio words”, we can generate new audio the same way we generate text — one word at a time!

Core Benefits: 1. **Audio becomes like text:** Can use the same algorithms that work for language 2. **Massive compression:** 1 second of raw audio (24,000 numbers) becomes ~75 tokens 3. **Controllable generation:** Can edit audio by changing specific tokens

1.6.3.4 How Neural Audio Codecs Work (Detailed Process)

The Multi-Level Description System: Think of describing a song at multiple levels of detail:

```
Level 1 (Coarse): "Upbeat rock song with vocals"
Level 2 (Medium): "Electric guitar, drums, male voice singing"
Level 3 (Fine): "Guitar with distortion, hi-hat cymbals, breathy vocals"
Level 4 (Finest): "Specific reverb on guitar, exact drum timing, vocal pitch bends"
...up to 8 levels of detail
```

Step-by-Step Process:

STEP 1: Compress the audio into features

- Raw audio (24,000 samples/second) → Compressed features (75 frames/second)
- Each frame captures the "essence" of a 320-sample chunk
- Like creating a summary of each tiny audio slice

STEP 2: Build descriptions at multiple detail levels (Residual Vector Quantization)

- Start with the compressed features
- Find the closest match from Codebook 1 (coarse patterns)
- Record the match ID (token) and see what's left over (residual)
- Find the closest match for the residual from Codebook 2 (medium patterns)
- Continue for 8 levels total, each capturing finer detail

STEP 3: Final result

- 8 sequences of token IDs (one per detail level)
- Total: $\sim 75 \text{ tokens/second} \times 8 \text{ levels} = 600 \text{ tokens per second}$
- Each token ID refers to a learned audio pattern

STEP 4: Reconstruction

- Look up each token in its corresponding codebook
- Add all 8 levels of detail back together
- Decode the combined features back to raw audio

1.6.3.5 Technical Terms Explained

Codebook: A learned dictionary of audio patterns. Think of it as a palette of 1,024 audio “colors” that can be mixed to recreate any sound.

Residual Vector Quantization (RVQ): A technique for describing audio at multiple levels of detail. Like having multiple description languages for the same audio — from coarse to fine.

Residual: What’s left over after you subtract the current approximation. If you describe a sound as “piano note” but it’s actually “piano note with reverb”, the reverb is the residual.

Frame Rate: How many audio snapshots per second. 75 frames/second means we analyze and describe audio in ~ 13 -millisecond chunks.

Bandwidth: How much information we keep. 6 kbps means 6,000 bits of information per second — much less than raw audio (384,000 bits/second).

1.6.3.6 Why This Multilevel Approach Works

The Painting Analogy: - **Level 1:** Rough sketch with basic shapes and colors - **Level 2:** Add major details and shading

- **Level 3:** Add texture and fine lines - **Level 8:** Add final highlights and micro-details

Each level adds more precision without affecting the previous levels. This lets the system balance between audio quality and compression efficiency.

Concrete Example — Codec Tokenization:

```

from encodec import EncodecModel
from encodec.utils import convert_audio
import torchaudio
import torch

# Load Encodec model (24kHz, 8 codebooks)
model = EncodecModel.encodec_model_24khz()
model.set_target_bandwidth(6.0) # 6 kbps

# Load audio
wav, sr = torchaudio.load("music.wav")
wav = convert_audio(wav, sr, model.sample_rate, model.channels)
wav = wav.unsqueeze(0) # Add batch dimension

# Encode to discrete tokens
with torch.no_grad():
    encoded_frames = model.encode(wav)

# Extract tokens
# encoded_frames is a list of (codes, scale) tuples
codes = encoded_frames[0][0] # (1, 8, T) — 8 codebook levels × T time frames
print(f"Audio tokens shape: {codes.shape}")
# For 5 seconds of audio: (1, 8, 375) — 75 frames/sec × 5 sec × 8 codebooks

print(f"Token IDs (codebook 1, first 10 frames): {codes[0, 0, :10]}")
# Output: tensor([ 42, 1337, 891, 4200, 567, 123, 890, 2345, 678, 901])

print(f"Codebook size: {model.quantizer.bins}") # 1024 entries per codebook
print(f"Total unique tokens: {model.quantizer.bins * 8}") # 8192

# Decode back to audio
with torch.no_grad():
    decoded_wav = model.decode(encoded_frames)

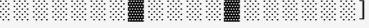
print(f"Reconstructed audio shape: {decoded_wav.shape}")
torchaudio.save("reconstructed.wav", decoded_wav.squeeze(0), model.sample_rate)

```

Visualization — Residual Vector Quantization (RVQ):

Original audio features z :  (continuous)

Level 1 quantization (coarse):  tokens: [42, 1337, 891, ...]
Residual (what's left): 

Level 2 quantization (detail):  tokens: [7, 234, 89, ...]
Residual (finer detail): 

...

Level 8 quantization (finest):  tokens: [1, 0, 3, ...]

Reconstruction = sum of all 8 quantized levels $\approx$ original

1.6.3.7 Method C: Music Generation Tokenization (Suno, MusicGen)

Used by: **Suno, MusicGen, Jukebox, MusicLM, Stable Audio**

Music generation systems tokenize audio differently depending on the use case:

1. MusicGen (Meta) — Codec-Based Music Generation:

```

from audiocraft.models import MusicGen
import torch

# Load model
model = MusicGen.get_pretrained("facebook/musicgen-medium")
model.set_generation_params(duration=10) # 10 seconds

# Generate from text description
descriptions = ["An upbeat electronic dance track with heavy bass"]
wav = model.generate(descriptions)
# wav shape: (1, 1, 320000) — 10 sec at 32kHz

# Under the hood:
# 1. Text → T5 encoder → text embeddings
# 2. Transformer decoder generates Encodec tokens autoregressively
# 3. Encodec decoder converts tokens → waveform
# 4. Uses "delay pattern" to handle 4 codebook levels efficiently

# The Transformer generates tokens like:
# Codebook 1: [42, 891, 1337, 567, ...] (melody/harmony — coarse)
# Codebook 2: [7, 234, 89, 456, ...] (timbre details)
# Codebook 3: [1, 12, 45, 78, ...] (texture)
# Codebook 4: [0, 3, 1, 5, ...] (fine details)

```

2. Music Composition Tokenization (MIDI-like):

For music understanding and composition (distinct from waveform generation), a more structured tokenization is used:

```

# Symbolic music tokenization (MidiTok library)
# Used by music analysis, composition, and interpretation systems

from miditok import REMI, TokenizerConfig

config = TokenizerConfig(
    num_velocities=32,
    use_chords=True,
    use_programs=True
)
tokenizer = REMI(config)

# Abstract token vocabulary for music:
# Position tokens: Bar_0, Position_0, Position_4, ...
# Pitch tokens: Pitch_60 (middle C), Pitch_72 (C5), ...
# Duration tokens: Duration_0.5 (eighth note), Duration_1.0 (quarter), ...
# Velocity tokens: Velocity_80 (medium), Velocity_120 (loud), ...
# Program tokens: Program_0 (piano), Program_25 (guitar), ...
# Chord tokens: Chord_C:maj, Chord_Am:min, ...

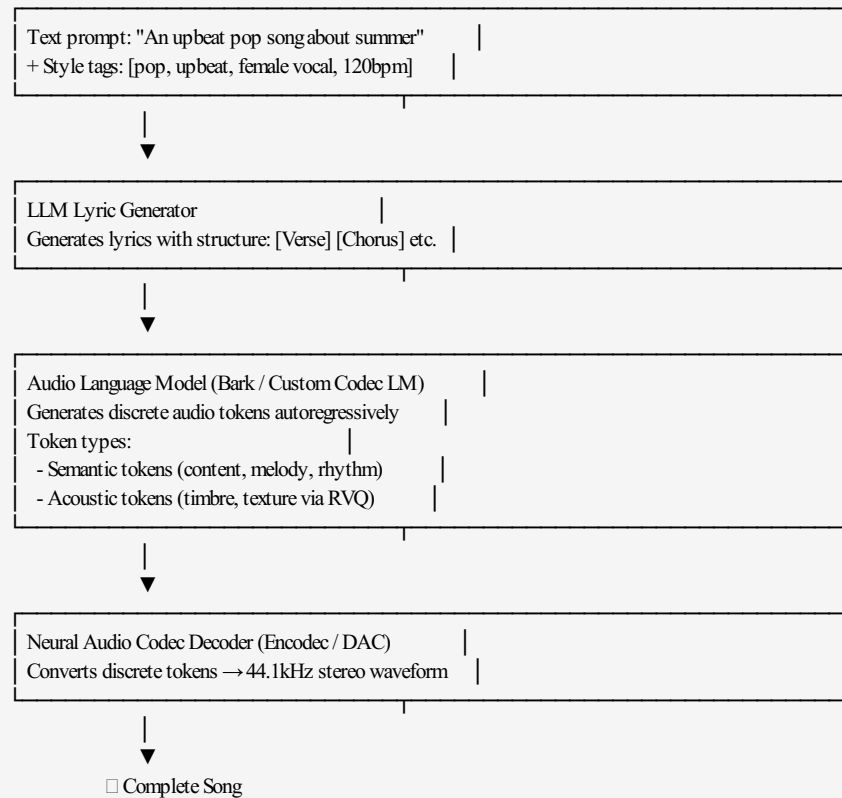
# Example tokenization of a simple melody:
# C4 quarter note, E4 quarter note, G4 half note
token_sequence = [
    "Bar_0",
    "Position_0", "Pitch_60", "Velocity_80", "Duration_1.0", # C4
    "Position_4", "Pitch_64", "Velocity_85", "Duration_1.0", # E4
    "Position_8", "Pitch_67", "Velocity_90", "Duration_2.0", # G4
    "Bar_1",
    # ... next bar
]

```

3. Suno-Style Full Song Generation:

Suno generates complete songs (vocals + instruments) using a pipeline:

Abstract Architecture:



1.6.4 Speech Synthesis (Text-to-Speech) Tokenization

Used by: VALL-E, Bark, Tortoise TTS, XTTS, Piper

Abstract — TTS Pipeline:

Text → Text Tokens → TTS Model → Audio Tokens (Codec) → Waveform

Specifically:

"Hello world" → [15339, 1917] → Transformer → [42, 891, 1337, ...] × 8 codebooks → □

Concrete — Bark TTS Example:

```
from bark import generate_audio, preload_models, SAMPLE_RATE
import scipy.io.wavfile

# Load models
preload_models()

# Generate speech (text → semantic tokens → coarse acoustic → fine acoustic → wav)
text = "Hello! This is a test of neural speech synthesis."
audio_array = generate_audio(text)

# Save
scipy.io.wavfile.write("speech_output.wav", SAMPLE_RATE, audio_array)

# Under the hood, Bark processes:
# 1. Text tokens (BPE): "Hello! This is..." → [token_ids]
# 2. Semantic model: text_tokens → semantic_tokens (content meaning)
# 3. Coarse acoustic model: semantic_tokens → coarse_codec_tokens (prosody, speaker)
# 4. Fine acoustic model: coarse_tokens → fine_codec_tokens (quality detail)
# 5. Encodec decoder: codec_tokens → waveform
```

1.7 Cross-Modal Tokenization

1.7.1 CLIP — Bridging Text and Image Tokens

CLIP (Contrastive Language-Image Pre-training) creates a **shared embedding space** for text and image tokens:

Image path: Image → ViT Patch Tokenization → [CLS, P1, ..., P196] → Image Embedding (512-dim)
Text path: Text → BPE Tokenization → [T1, T2, ..., Tn] → Text Embedding (512-dim)

Both embeddings live in the same 512-dim space!
Cosine similarity measures text-image match.

```
import torch
from transformers import CLIPProcessor, CLIPModel

model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

# Process image + text together
inputs = processor(
    text=["a photo of a cat", "a photo of a dog"],
    images=[cat_image],
    return_tensors="pt",
    padding=True
)

# Get embeddings
outputs = model(**inputs)
image_embeds = outputs.image_embeds # (1, 512)
text_embeds = outputs.text_embeds # (2, 512)

# Compare: which text matches the image?
similarity = torch.cosine_similarity(image_embeds, text_embeds)
print(f"Cat similarity: {similarity[0]:.3f}") # High (~0.28)
print(f"Dog similarity: {similarity[1]:.3f}") # Lower (~0.20)
```

1.7.2 Multimodal Tokens in GPT-4o / Gemini

Modern multimodal models unify all modalities into a single token sequence:

User message: "What's in this image? [IMAGE] Also, transcribe this audio [AUDIO]"

Internal token sequence:
[SYS_TOKENS] [TEXT: "What's in this image?"] [IMG_START] [P1, P2, ..., P256] [IMG_END]
[TEXT: "Also, transcribe this audio"] [AUDIO_START] [A1, A2, ..., A500] [AUDIO_END]

→ Transformer processes ALL tokens in one forward pass
→ Output: text tokens describing image + audio transcription

1.8 Comparison Matrix

1.8.1 Text Tokenizers

Algorithm	Vocabulary	Training	Used By	Multilingual	Speed
BPE	32K–200K	Frequency merging	GPT-4, LLaMA 3, Mistral	Good (byte-level)	Fast
WordPiece	30K–110K	Likelihood scoring	BERT, DistilBERT	Good	Fast
Unigram	32K–250K	Probabilistic pruning	T5, mBART, ALBERT	Excellent	Medium
tiktoken	100K–200K	BPE + regex	GPT-3.5/4/4o	Good	Very Fast

1.8.2 Image Tokenizers

Method	Token Count	Compression	Reversible	Used By	Quality
ViT Patches	196–576	48–8×	No (not generative)	ViT, CLIP, GPT-4V	N/A (analysis)
VQ-VAE (8K)	256–1024	48–192×	~Lossy	DALL-E 1	Medium
VQ-GAN (16K)	256–1024	48–192×	~Good	Parti, Muse	High
VAE Latent	64×64×4	48×	~Very Good	Stable Diffusion	Very High

1.8.3 Audio Tokenizers

Method	Tokens/sec	Codebooks	Quality	Used By	Use Case
Mel Spectrogram	100 frames	N/A (continuous)	N/A	Whisper, ASR	Speech recognition
Encodec (1.5kbps)	75	2	Moderate	Bark (low quality)	Voice messaging
Encodec (6kbps)	75	8	High	MusicGen, VALL-E	Music generation
Encodec (24kbps)	75	32	Very High	High-fidelity audio	Production audio
DAC	86	9	Very High	Stable Audio	Music production
SoundStream	75	8–12	High	Google AudioLM	Speech + music
MIDI Tokens	Variable	1	Symbolic	Composition AI	Sheet music

1.9 References

1.9.1 Academic Papers

1. **BPE**: Sennrich, R., Haddow, B., & Birch, A. (2016). “Neural Machine Translation of Rare Words with Subword Units.” ACL.
2. **WordPiece**: Schuster, M., & Nakajima, K. (2012). “Japanese and Korean voice search.” ICASSP.
3. **SentencePiece**: Kudo, T., & Richardson, J. (2018). “SentencePiece: A simple and language independent subword tokenizer.” EMNLP.
4. **ViT**: Dosovitskiy, A. et al. (2020). “An Image is Worth 16x16 Words.” ICLR.
5. **VQ-VAE**: van den Oord, A. et al. (2017). “Neural Discrete Representation Learning.” NeurIPS.
6. **DALL-E**: Ramesh, A. et al. (2021). “Zero-Shot Text-to-Image Generation.” ICML.
7. **Encodec**: Défossez, A. et al. (2022). “High Fidelity Neural Audio Compression.” arXiv.
8. **Whisper**: Radford, A. et al. (2022). “Robust Speech Recognition via Large-Scale Weak Supervision.” arXiv.
9. **MusicGen**: Copet, J. et al. (2023). “Simple and Controllable Music Generation.” NeurIPS.
10. **CLIP**: Radford, A. et al. (2021). “Learning Transferable Visual Models From Natural Language Supervision.” ICML.

1.9.2 Tools & Libraries

Tool	Purpose	URL
tiktoken	OpenAI BPE tokenizer	https://github.com/openai/tiktoken
SentencePiece	Language-agnostic tokenizer	https://github.com/google/sentencepiece
Hugging Face Tokenizers	Fast tokenizer library	https://github.com/huggingface/tokenizers
MidiTok	Music tokenization	https://github.com/Natooz/MidiTok
Encodec	Neural audio codec	https://github.com/facebookresearch/encodec
torchaudio	Audio processing	https://pytorch.org/audio/
CLIP	Vision-language model	https://github.com/openai/CLIP
timm	Vision transformers	https://github.com/huggingface/pytorch-image-models